You must provide one and only one argument when you invoke this script. Otherwise, dtrace will not execute. DTrace is dynamic: It shows events as they happen. If you probe a process that is not busy, you see a blank line until a matching event occurs. Try the command with a busy process such as a web browser process.

5.2.5 Using Aggregate Functions

If you specified a busy process, you probably received a very large volume of output. The D language aggregate construct helps you manage this output. Aggregations collect the output in tables in memory and output a summary. Aggregations have the following form:

```
@name[table index(es)] =aggregate_function()
```

The following is an example of an aggregate construct:

```
@count_table[probefunc] = count() ;
```

Add this construct to the action area of your script

```
#!/usr/sbin/dtrace -s
pid$1:::entry
{
        @count_table[probefunc] = count() ;
}
```

When you run the modified script, you see output similar to the following. Most of the data is omitted from this sample output.

```
# ./myDscript.d 23
dtrace: script './myDscript' matched 7954 probes
^C
  __clock_gettime                   5
  ioctl                             20
  mutex_lock                       767
  sigon                           1554
```

This script collects information into a table until you enter Ctrl-C. When you enter Ctrl-C, DTrace outputs the data that was collected during that time. The output table lists each function only one time, with the number of times that function was entered, from fewer to greater number of times entered. The variable probefunc is a built-in variable that holds the name